

UD7 – Distribución de Aplicaciones

Contenido

1. Componentes de una aplicación. Empaquetado	1
2. Instaladores. Paquetes autoinstalables	2
3. Herramientas para crear paquetes de instalación	2
4. Parámetros de la instalación y Personalización del asistente	4
5. Asistentes de instalación y desinstalación. Interacción con el usuario	5
6. Ficheros firmados digitalmente	5
7. Instalación de aplicaciones desde un servidor web. Descarga y ejecución	6

1. Componentes de una aplicación. Empaquetado

Una aplicación lista para distribuir no es solo el código que hemos escrito. Debemos empaquetar todos los elementos necesarios en un conjunto organizado y optimizado para que funcione correctamente en cualquier dispositivo o equipo del usuario final.

Componentes que se deben incluir en el paquete:

- Código compilado (AAB o APK en Android, o ejecutable en WPF).
- Recursos de interfaz: layouts XML, drawables, temas de Material Design 3 e iconos adaptativos (Android), y recursos XAML, imágenes y estilos (WPF).
- Archivos de configuración: [AndroidManifest.xml](#), [build.gradle.kts](#), [strings.xml](#) (para internacionalización), [App.config](#) y archivos de recursos en WPF.
- Dependencias y bibliotecas (Gestor Gradle en Android y paquetes NuGet en WPF).
- Archivos de datos (bases de datos locales con Room o SQLite, archivos JSON de configuración).
- Documentación generada con Dokka (Tema 6.1) y el archivo de notas de versión (changelog).
- Metadatos de versión y firma digital.

```
# Changelog - Jugando con botones
```

```
## v1.0.0 - 27/03/2026
```

- Primera versión estable.
- Botones dinámicos en ScrollView.
- Toast al pulsar cada botón.

```
## v0.9.0 - 20/03/2026
```

- Versión beta.

Ejemplo de changelog.md

Ejemplos prácticos:

- **Android con Kotlin:** Se genera un **Android App Bundle (AAB)**, el formato recomendado por Google. Este paquete permite que Google Play cree versiones optimizadas para cada dispositivo, reduciendo el tamaño de la descarga.
- **WPF con C#:** Se incluyen todos los archivos de la carpeta **bin/Release/net8.0-windows**, las carpetas de recursos y el instalador que crearemos.

2. Instaladores. Paquetes autoinstalables

Un paquete autoinstalable es un archivo único que permite instalar la aplicación de forma sencilla, sin que el usuario necesite conocimientos técnicos.

Existen tres modos de instalación que debemos conocer y utilizar:

- **Instalación típica:** Es la opción recomendada para la mayoría de los usuarios. Instala los componentes esenciales.
- **Instalación completa:** Incluye componentes adicionales como tutoriales, ejemplos de datos o documentación completa.
- **Instalación personalizada:** Permite al usuario elegir qué componentes instalar.

En Android con Kotlin: No existe un asistente gráfico tradicional. El sistema Android Package Installer se encarga de la instalación. Preparamos un AAB para Google Play o un APK firmado para distribución directa.

En WPF con C#: Creamos un asistente gráfico (wizard) completo y profesional mediante herramientas externas.

3. Herramientas para crear paquetes de instalación

Podemos crear los paquetes de dos formas principales:

Android con Kotlin

- Desde el entorno de desarrollo: Android Studio → Build → Generate Signed Bundle/APK.
- Herramientas externas: bundletool, apksigner y zipalign.

WPF con C#

- Desde el entorno de desarrollo: Visual Studio → Publish (ClickOnce o carpeta Release) o empaquetado MSIX.
- Herramientas externas: **Inno Setup** (la más utilizada en el módulo), WiX Toolset o Advanced Installer (versión gratuita).

Plataforma	Desde el entorno de desarrollo	Herramientas externas más utilizadas
Android con Kotlin	Android Studio → Build → Generate Signed Bundle/APK	bundletool, apksigner, zipalign
WPF con C#	Visual Studio → Publish (ClickOnce o carpeta Release) o MSIX	Inno Setup (la más usada en el módulo), WiX Toolset, Advanced Installer

Ejemplo práctico de script para Inno Setup (WPF)

```
[Setup]
AppName=FormularioEmpleados
AppVersion=1.0
DefaultDirName={autopf}\FormularioEmpleados
OutputDir=.
OutputBaseFilename=Setup_FormularioEmpleados
Compression=lzma
SolidCompression=yes
SetupIconFile=icono.ico
WizardImageFile=fondo.bmp

[Files]
Source: "bin\Release\*.exe"; DestDir: "{app}"; Flags: ignoreversion
Source: "bin\Release\*.dll"; DestDir: "{app}"; Flags: ignoreversion

[Tasks]
Name: "desktopicon"; Description: "Crear acceso directo en el escritorio";
GroupDescription: "Opciones adicionales"

[Icons]
Name: "{group}\FormularioEmpleados"; Filename: "{app}\TuProyecto.exe"
Name: "{userdesktop}\FormularioEmpleados"; Filename: "{app}\TuProyecto.exe"; Tasks:
desktopicon
```

Este script genera un instalador profesional con icono e imagen de fondo.

4. Parámetros de la instalación y Personalización del asistente

Durante la instalación podemos personalizar la experiencia para que sea más profesional y fácil de usar.

Elementos de personalización:

- Pantallas de bienvenida y final (usando imágenes del proyecto de Material Design o XAML).
- Selección de idioma (vinculada a la internacionalización).
- Directorio de instalación por defecto.
- Creación automática de accesos directos en el escritorio y en el menú Inicio (WPF).
- Diálogos de licencia y política de privacidad.
- Modos de instalación (típica, completa y personalizada).

En Android: La personalización principal se realiza en [Google Play Console](#) (descripciones, capturas de pantalla, vídeos y política de privacidad en varios idiomas).

5. Asistentes de instalación y desinstalación. Interacción con el usuario

El asistente de instalación debe ser claro, accesible y con buena usabilidad: barra de progreso, mensajes informativos y opción de cancelar en cualquier momento.

La **desinstalación** debe ser limpia:

- Eliminar todos los archivos de la aplicación.
- Borrar los accesos directos creados.
- Limpiar las entradas de registro (en WPF).

En Android el sistema operativo gestiona la desinstalación automáticamente.

Interacción con el usuario: Todos los mensajes deben estar en lenguaje claro, con soporte multilingüe y accesibilidad (alto contraste y tamaño de texto adecuado).

6. Ficheros firmados digitalmente

La firma digital garantiza que la aplicación no ha sido modificada y que proviene del desarrollador legítimo.

Android con Kotlin:

- Crear un keystore en Android Studio.
- Firmar el AAB o APK con **Build → Generate Signed Bundle/APK**.
- Recomendado: Activar **Google Play App Signing** (Google gestiona la clave de firma de forma segura).

WPF con C#:

- Firmar con certificado Code Signing (para pruebas puedes usar un certificado autofirmado).
- Firmar el instalador generado con Inno Setup o el archivo .exe/.msi.

Comando útil (Android):

Bash

```
apksigner sign --ks mi-keystore.jks --out app-release-signed.aab app-release.aab
```

Sin firma correcta, Google Play rechaza la aplicación y Windows muestra advertencias de seguridad.

7. Instalación de aplicaciones desde un servidor web. Descarga y ejecución

Esta es una forma muy habitual de distribuir aplicaciones en entornos empresariales (fuera de las tiendas oficiales).

Pasos para preparar la distribución web:

1. Generar el paquete final (AAB/APK firmado o instalador WPF).
2. Subirlo a un servidor web (GitHub Releases, servidor del centro, OneDrive, etc.).
3. Crear una página HTML sencilla que incluya:
 - Descripción clara de la aplicación.
 - Capturas de pantalla.
 - Enlace de descarga directa.
 - Código QR para facilitar la descarga desde el móvil.
4. Añadir instrucciones claras:
 - Android: “Permitir instalación de orígenes desconocidos”.
 - WPF: “Ejecutar como administrador”.